LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL [5] – [Relasi antar Kelas]



NAMA: Fauzi Romadhoni

NIM: 103132400025

KELAS: SISD-05-A

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Ringkasan mengenai konsep relasi kelas yaitu Asosiasi, Agregasi, dan Komposisi

- Asosiasi: Hubungan antara dua kelas terpisah yang terbentuk melalui objek pada tiap kelas, dimana suatu kelas dapat menggunakan method dari kelas lain contoh: Dosen menggunakan method dari objek Mahasiswa.
- Agregasi: Hubungan antara dua kelas yang lebih ketat, terdapat hubungan bagian dan keseluruhan tetapi bisa diciptakan sendiri-sendiri.
- Komposisi: Bentuk khusus dari agregasi dengan hubungan lebih kuat, di mana part (bagian) tidak akan ada tanpa adanya kelas whole (contoh: CPU, Monitor, dan Mouse adalah bagian dari Komputer, dan musnah jika Komputernya musnah).

B. Guided

1. [**Asosiasi**]

```
[// File: Mahasiswa.java
public class Mahasiswa {
   private String nim;
   private String nama;
   private int nilai;

   public void setNama(String nama) { this.nama = nama; }
   public void setNim(String nim) { this.nim = nim; }
   public String getNama() { return this.nama; }
   public String getNim() { return this.nim; }
   public void setNilai(int nilai) { this.nilai = nilai; }
   public int getNilai() { return this.nilai; }
}

// File: Dosen.java (Ini sebagai Main Class untuk di-run)
public class Dosen {
   private String kodeDosen;
   private String namaDosen;

   public void setKodeDosen(String kodeDosen) { this.kodeDosen = kodeDosen;
}
   public void setNamaDosen(String namaDosen) { this.namaDosen =
namaDosen; }
   public String getKodeDosen() { return this.kodeDosen; }
   public String getNamaDosen() { return this.namaDosen; }

   public void giveScore(Mahasiswa s, int nilai) {
      s.setNilai(nilai);
   }
   public int getScore(Mahasiswa s) {
```

```
        return s.getNilai();
    }

    public static void main(String[] args) {
        Mahasiswa m = new Mahasiswa();
        m.setNim("130118383");
        m.setNama("Budi");

        Dosen d = new Dosen();
        d.giveScore(m, 90);

        System.out.println("Nim: " + m.getNim());
        System.out.println("Nama: " + m.getNama());
        System.out.println("Nilai: " + d.getScore(m));
    }
}]
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ MODUL5 ---
Nim: 130118383
Nama: Budi
Nilai: 90
------------------------------------------------
BUILD SUCCESS
[                                                ]
```

2. [**Agregasi**]

```
[// File: Mahasiswa.java
public class Mahasiswa {
    private String NIM, Nama;
    public Mahasiswa(String no, String nm) {
        this.NIM = no;
        this.Nama = nm;
    }
    public String GetNIM() { return (NIM); }
    public String GetNama() { return (Nama); }
}

// File: Jurusan.java
public class Jurusan {
    private String KodeJurusan, NamaJurusan;
    private Mahasiswa[] Daftar = new Mahasiswa[10];
    private int JmlMhs = 0;

    public Jurusan(String kode, String nama) {
        this.KodeJurusan = kode;
        this.NamaJurusan = nama;
    }
    public void AddMahasiswa(Mahasiswa m) {
```

```
        this.Daftar[JmlMhs] = m;
        this.JmlMhs++;
    }
    public void DisplayMahasiswa() {
        int i;
        System.out.println("Kode Jurusan : " + this.KodeJurusan);
        System.out.println("Nama Jurusan: " + this.NamaJurusan);
        System.out.println("Daftar Mahasiswa :");
        for(i = 0; i < JmlMhs; i++) {
            System.out.println(Daftar[i].GetNIM() + " " + Daftar[i].GetNama());
        }
    }

    // Tambahan method main agar bisa di-run (opsional jika diminta)
    public static void main(String[] args) {
        Jurusan j = new Jurusan("IF", "Informatika");
        Mahasiswa m1 = new Mahasiswa("123", "Budi");
        Mahasiswa m2 = new Mahasiswa("124", "Andi");

        j.AddMahasiswa(m1);
        j.AddMahasiswa(m2);
        j.DisplayMahasiswa();
    }
}]
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ MODUL5 ---
Kode Jurusan : IF
Nama Jurusan: Informatika
Daftar Mahasiswa :
123 Budi
124 Andi
------------------------------------------------------------------------
[ BUILD SUCCESS                                                        ]
```

C. Unguided

1. Soal Unguided 1

   [Sebuah tim mengumpulkan suhu harian suatu kota selama satu minggu (7 hari) dari sensor otomatis. Data ini sudah tersedia sebagai array di main sebelum diserahkan untuk diolah.

   Pada salah satu hari, sensor gagal mengirim data. Hari yang gagal terekam ditandai dengan sebuah nilai penanda pada array, karena suhu asli tidak mungkin sebesar itu. Nilai penanda ini sama untuk semua data suhu yang diolah kapan pun dan oleh siapa pun — bukan sesuatu yang berbeda-beda tiap kali data dibuat — sehingga sebaiknya

ditulis satu kali saja sebagai aturan tetap milik class, bukan diketik ulang sebagai angka -1.0 di setiap method yang membutuhkannya.
Buatlah satu class yang bertugas mengolah data suhu ini — memeriksa apakah ada data kosong, mengisinya, dan menghasilkan statistik dasar dari data yang sudah bersih.
]

Jawaban
: ItemPesanan

```
[package com.mycompany.cafeapp;

public class ItemPesanan {
   private String namaItem;
   private RincianHarga rincian;

   public ItemPesanan(String namaItem, double hargaDasar) {
      this.namaItem = namaItem;
      // Komposisi: RincianHarga dibuat langsung di dalam constructor
      this.rincian = new RincianHarga(hargaDasar);
   }

   public String getNamaItem() {
      return namaItem;
   }

   public RincianHarga getRincian() {
      return rincian;
   }
}]
```

; Kasir

```
        [package com.mycompany.cafeapp;

        public class Kasir {
           private String namaKasir;

           public Kasir(String namaKasir) {
              this.namaKasir = namaKasir;
           }

           public String getNamaKasir() {
              return namaKasir;
           }

           // Asosiasi: Pesanan hanya diterima lewat parameter, tidak disimpan
        sebagai field
```

```
    public void prosesPembayaran(Pesanan pesanan) {
        double totalSubtotal = 0;
        double totalPajak = 0;
        ItemPesanan[] items = pesanan.getDaftarItem();

        for (int i = 0; i < pesanan.getJumlahItem(); i++) {
            RincianHarga r = items[i].getRincian();
            totalSubtotal += r.getHargaDasar();
            totalPajak += r.getPajak();
        }

        double totalBayar = totalSubtotal + totalPajak;

        System.out.println("=== STRUK PEMBAYARAN KASIR ===");
        System.out.println("Kasir Bertugas : " + namaKasir);
        System.out.println("Pesanan        : Meja " +
pesanan.getNomorMeja());
        System.out.println("Jumlah Item    : " + pesanan.getJumlahItem());
        System.out.println("Total Subtotal : Rp " + totalSubtotal);
        System.out.println("Total Pajak    : Rp " + totalPajak);
        System.out.println("TOTAL BAYAR    : Rp " + totalBayar);
        System.out.println("==============================");
    }
}]
```

; Main

```
[package com.mycompany.cafeapp;

public class Main {
    public static void main(String[] args) {
        // 1. Buat Kasir
        Kasir kasir = new Kasir("Budi");

        // 2. Buat 3 ItemPesanan secara independen
        ItemPesanan item1 = new ItemPesanan("Iced Americano", 18000);
        ItemPesanan item2 = new ItemPesanan("Beef Croissant", 25000);
        ItemPesanan item3 = new ItemPesanan("Matcha Latte", 22000);

        // 3. Buat Pesanan untuk Meja 4 (kapasitas 5 item)
        Pesanan pesanan = new Pesanan(4, 5);

        // 4. Masukkan item ke pesanan (Agregasi)
        pesanan.tambahItem(item1);
        pesanan.tambahItem(item2);
        pesanan.tambahItem(item3);
```

```
            // 5. Tampilkan pesanan
            pesanan.tampilkanPesanan();

            // 6. Proses pembayaran (Asosiasi)
            kasir.prosesPembayaran(pesanan);
        }
}]
```

; Pesanan

```
    [package com.mycompany.cafeapp;

    public class Pesanan {
        private int nomorMeja;
        private ItemPesanan[] daftarItem;
        private int jumlahItem;

        public Pesanan(int nomorMeja, int kapasitas) {
            this.nomorMeja = nomorMeja;
            this.daftarItem = new ItemPesanan[kapasitas];
            this.jumlahItem = 0;
        }

        public void tambahItem(ItemPesanan item) {
            if (jumlahItem < daftarItem.length) {
                daftarItem[jumlahItem] = item;
                jumlahItem++;
            } else {
                System.out.println("Pesanan penuh, item " + item.getNamaItem() +
    " tidak dapat ditambahkan.");
            }
        }

        public void tampilkanPesanan() {
            System.out.println("=== DAFTAR PESANAN CAFE ===");
            System.out.println("Nomor Meja: " + nomorMeja);
            for (int i = 0; i < jumlahItem; i++) {
                ItemPesanan item = daftarItem[i];
                RincianHarga r = item.getRincian();
                System.out.println((i + 1) + ". " + item.getNamaItem()
                        + " - Rp " + r.getHargaTotal()
                        + " (Harga: " + r.getHargaDasar()
                        + ", Pajak: " + r.getPajak() + ")");
            }
        }

        public int getNomorMeja() {
```

```
        return nomorMeja;
    }

    public ItemPesanan[] getDaftarItem() {
        return daftarItem;
    }

    public int getJumlahItem() {
        return jumlahItem;
    }
}]
```

; RincianHarga

```
    [package com.mycompany.cafeapp;

    public class RincianHarga {
        private double hargaDasar;
        private double pajak;

        public RincianHarga(double hargaDasar) {
            this.hargaDasar = hargaDasar;
            this.pajak = hargaDasar * 0.10;
        }

        public double getHargaDasar() {
            return hargaDasar;
        }

        public double getPajak() {
            return pajak;
        }

        public double getHargaTotal() {
            return hargaDasar + pajak;
        }
}]
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ MODUL5 ---
=== DAFTAR PESANAN CAFE ===
Nomor Meja: 4
1. Iced Americano - Rp 19800.0 (Harga: 18000.0, Pajak: 1800.0)
2. Beef Croissant - Rp 27500.0 (Harga: 25000.0, Pajak: 2500.0)
3. Matcha Latte - Rp 24200.0 (Harga: 22000.0, Pajak: 2200.0)
=== STRUK PEMBAYARAN KASIR ===
Kasir Bertugas : Budi
Pesanan        : Meja 4
Jumlah Item    : 3
Total Subtotal : Rp 65000.0
Total Pajak    : Rp 6500.0
TOTAL BAYAR    : Rp 71500.0
==============================
------------------------------------------------------------------------
[  BUILD SUCCESS                                                       ]
```

Penjelasan :

[Proyek ini bertujuan untuk melatih penerapan tiga jenis relasi antar kelas dalam Pemrograman Berorientasi Objek (PBO) melalui studi kasus sistem pesanan di sebuah cafe. Program harus menggunakan array standar bertipe tetap (tanpa ArrayList),]

D. Kesimpulan

[Asosiasi kelas yang sekadar menggunakan method kelas lain, Agregasi hubungan bagian-keseluruhan yang kelasnya dapat diciptakan sendiri-sendiri, dan Komposisi hubungan bagian-keseluruhan yang sangat erat dan saling bergantung.]